

Manual de usuario

Contacto 3xtrat3rr3str3D

1. Introducción

Contacto 3xtrat3rr3str3D es una aplicación de escritorio que permite escribir, analizar y compilar programas en tres lenguajes de programación propios:

- .y (Y?) — lenguaje de funciones y estructuras.
- .z (Zetariano) — lenguaje orientado a objetos.
- .pig (Pig Latin) — lenguaje principal que integra los tres y contiene el programa ejecutable.

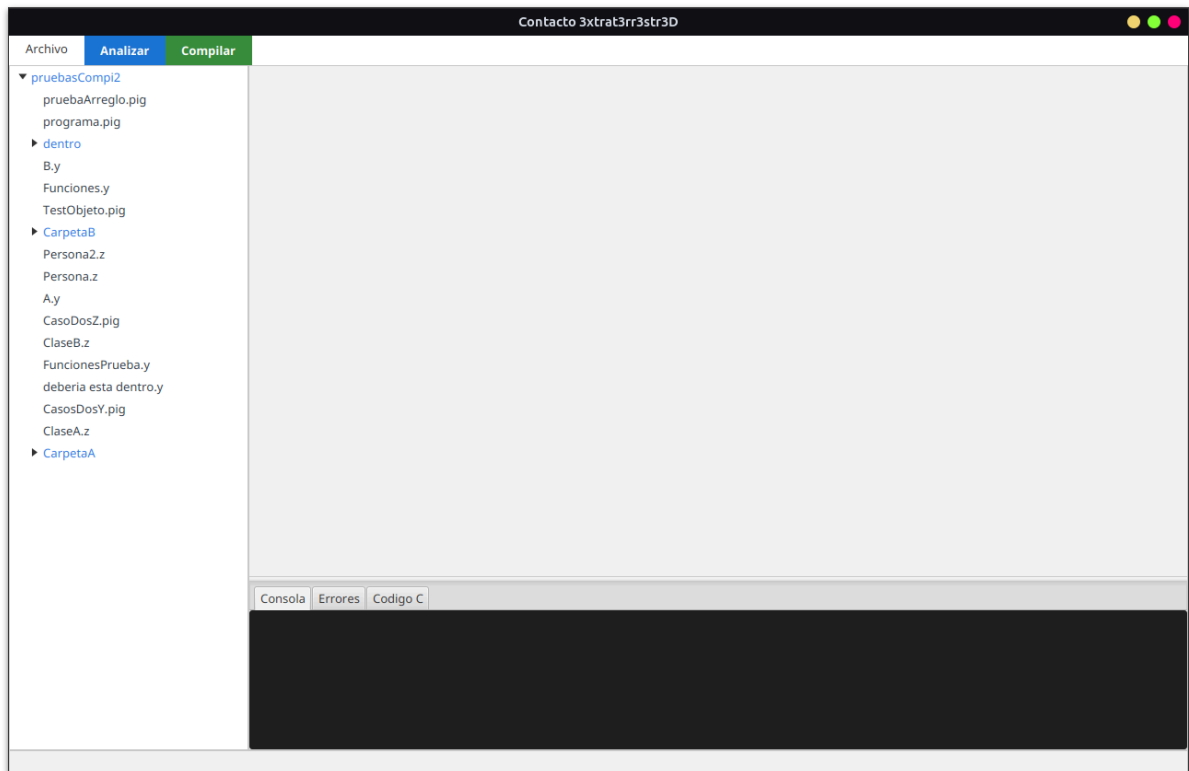
La aplicación valida el código, genera código C a partir de él, lo compila con GCC y produce un ejecutable que se puede correr desde la terminal del sistema.

2. Requisitos del sistema

Requisito	Detalle
Sistema operativo	Windows, Linux o macOS
Java	Versión 21 o superior
GCC	Instalado y accesible en el PATH
Verificar que tienes GCC: Abre una terminal y escribe: bash gcc --version Debe responder con la versión instalada. Si no responde, GCC no está instalado y la compilación fallará.	

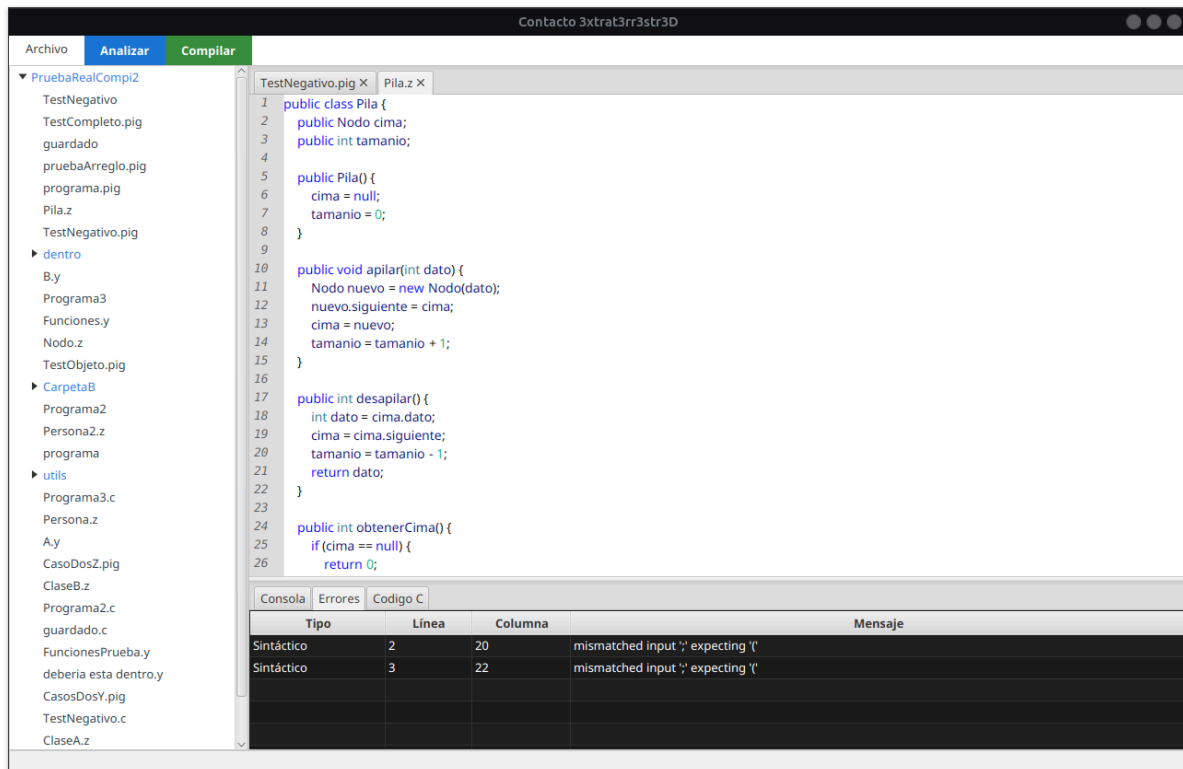
3. Inicio de la aplicación

Al abrir la aplicación, verás una ventana dividida en varias zonas:



Zona	Qué hace
Menú superior	Contiene las acciones principales
Árbol de trabajo	Muestra las carpetas y archivos abiertos
Pestañas de edición	Un editor por cada archivo abierto
Panel de salida	Muestra mensajes, errores y el código C generado

4. Gestión de archivos



4.1 Abrir un archivo

Menú: Archivo → Abrir archivo...

Atajo: Ctrl + O

Se abre un diálogo donde puedes seleccionar un archivo .y, .z o .pig.

4.2 Abrir una carpeta

Menú: Archivo → Abrir carpeta...

Atajo: Ctrl + Shift + O

Abre una carpeta completa en el árbol de trabajo. Los archivos aparecen en el panel izquierdo y puedes hacer clic en cualquiera para abrirlo en el editor.

4.3 Crear un archivo nuevo

Menú: Archivo → Nuevo archivo

Atajo: Ctrl + N

Se abre un diálogo donde eliges el lenguaje:

- Y? → crea un archivo .y
- Zetariano → crea un archivo .z
- Pig Latin → crea un archivo .pig

Se abre una pestaña nueva con un editor vacío.

4.4 Guardar un archivo

Menú: Archivo → Guardar

Atajo: Ctrl + S

Guarda el archivo actual. Si nunca se ha guardado, se abre un diálogo para elegir dónde y con qué nombre.

4.5 Guardar como

Menú: Archivo → Guardar como...

Atajo: Ctrl + Shift + S

Guarda el archivo con otro nombre o en otra ubicación.

4.6 Descargar un archivo o carpeta

Menú: Archivo → Descargar archivo... o Descargar carpeta...

Copia un archivo o carpeta a otra ubicación del sistema.

4.7 Salir

Menú: Archivo → Salir

Atajo: Ctrl + Q

Cierra la aplicación. Si hay cambios sin guardar, se pierden.

5. El editor de código

Cada archivo abierto aparece en una pestaña dentro del área de edición.

5.1 Características del editor

- Números de línea en el margen izquierdo.
- Resaltado de sintaxis automático según el lenguaje:
 - .y: azul para palabras reservadas, verde para tipos.
 - .z: colores tipo Java.
 - .pig: colores propios del lenguaje.
- Deshacer/Rehacer con Ctrl + Z / Ctrl + Y.
- Cortar/Copiar/Pegar con Ctrl + X / Ctrl + C / Ctrl + V.

5.2 Pestaña modificada

Si modificas un archivo sin guardarlo, la pestaña muestra un asterisco * delante del nombre:
text

* Programa.pig

Al guardar, el asterisco desaparece.

5.3 Cerrar una pestaña

Haz clic en la X de la pestaña. Si hay cambios sin guardar, se pierden.

6. Analizar un archivo

El análisis verifica que el código esté léxica, sintáctica y semánticamente correcto, sin generar código C.

6.1 Cómo analizar

Menú: Analizar

Atajo: F5

Importante: el archivo debe estar guardado antes de analizar. Si no lo está, aparece un error.

6.2 Qué verifica el análisis

Fase	Qué revisa
Léxico	Caracteres inválidos, tokens mal formados
Sintáctico	Estructura del código (paréntesis, llaves, indentación)
Semántico	Tipos, variables declaradas, funciones existentes, flujo de control

6.3 Resultado del análisis

- Si hay errores: aparecen en la pestaña Errores del panel de salida. Cada error muestra:
 - Tipo (Léxico, Sintáctico, Semántico)
 - Línea y columna
 - Mensaje descriptivo
- Si no hay errores: en la pestaña Consola aparece el mensaje "Análisis exitoso. Sin errores."

7. Compilar un archivo

La compilación analiza el archivo, genera código C, lo compila con GCC y produce un ejecutable.

7.1 Requisito

Solo funciona con archivos .pig. Los archivos .y y .z no se compilan solos porque no tienen main.

Si intentas compilar un .y o .z, aparece un mensaje indicando que solo se compila .pig.

7.2 Cómo compilar

Menú: Compilar

Atajo: F6

7.3 Pasos de la compilación

1. Se abre un diálogo para elegir dónde guardar el archivo .c generado.
 - Por defecto se sugiere el nombre del .pig con extensión .c (ej. Programa.pig → Programa.c).

- La primera vez se abre en la carpeta del .pig.
 - Las siguientes veces se abre en la última carpeta usada.
- 2. Se genera el código C y se escribe en el archivo elegido.
- 3. Se llama a GCC para compilarlo.
- 4. El ejecutable se genera al lado del .c con el mismo nombre sin extensión (ej. Programa.c → Programa).

7.4 Resultado de la compilación

- Si todo sale bien:
 - En la pestaña Código C se muestra el código generado.
 - En la pestaña Consola aparece:
text

Compilación exitosa.

- Ejecutable: /ruta/completa/al/ejecutable
 - Si hay errores semánticos: no se genera C. Se muestran los errores en la pestaña Errores.
 - Si GCC falla: se muestra el código C de todas formas y en la Consola aparece el error de GCC.
-

8. Ejecutar el programa compilado

La aplicación no ejecuta el programa desde el UI. Debes ejecutarlo desde la terminal del sistema.

8.1 Cómo ejecutarlo

1. Abre una terminal.
2. Navega hasta la carpeta donde se generó el ejecutable.
3. Ejecuta:
Linux / macOS:
 4. bash
 5. ./ProgramaWindows:
 6. cmd
 7. Programa.exe

8.2 Aviso sobre << (lectura)

Si el .pig usa << (lectura de consola), el programa pedirá datos al usuario. En la terminal puedes escribir y pulsar Enter para continuar.

Ejemplo:

text

Ingresa tu nombre:

> Juan

Hola, Juan

9. Panel de salida

El panel de salida tiene 3 pestañas:

9.1 Consola

Muestra mensajes del proceso:

- "Analizando Programa.pig como Pig Latin"
- "Análisis exitoso. Sin errores."
- "Compilando Programa.pig hacia /ruta/Programa.c"
- "Compilación exitosa."
- "Ejecutable: /ruta/Programa"
- Mensajes de error de GCC.

9.2 Errores

Muestra una tabla con todos los errores:

Tipo	Línea	Columna	Mensaje
Sintáctico	5	12	missing 'finis' at ';'
Semántico	10	3	Variable 'x' no declarada
Semántico	15	1	Tipo incompatible en asignación

Haz clic en cualquier fila para ver el detalle.

9.3 Código C

Muestra el código C generado tras una compilación exitosa. Puedes copiarlo si quieres revisarlo o llevarlo a otro editor.

10. Los tres lenguajes

10.1 Lenguaje .y (Y?)

Lenguaje de funciones y estructuras, con sintaxis basada en indentación (como Python).

Ejemplo:

y

%estructuras

estructura Punto:

 entero x

 entero y

%funciones

definir sumar(entero a, entero b) -> entero:

retornar a + b

definir main():

Punto p

p.x = 10

p.y = 20

imprimir(p.x + p.y)

Características:

- Sin variables globales.
- Estructuras anidadas.
- Arreglos y matrices.
- Ciclos para, mientras, hacer.
- Condicionales si / sino / contrario.
- Selección elegir / caso / siempre.
- imprimir(...) y leer().

10.2 Lenguaje .z (Zetariano)

Lenguaje orientado a objetos, con sintaxis basada en llaves (como Java).

Ejemplo:

java

```
public class Persona {  
    String nombre;  
    int edad;  
  
    public Persona(String nombreParametro, int edadParametro) {  
        nombre = nombreParametro;  
        edad = edadParametro;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void saludar() {  
        println("Hola, soy " + nombre);  
    }  
}
```

Características:

- Una sola clase por archivo.
- El archivo debe llamarse como la clase (Persona.z).
- Constructores sobrecargables.
- Métodos sobrecargables.
- Atributos públicos (sin public).
- Métodos y constructores con public.
- Arreglos 1D y 2D.
- if / else if / else, switch / case / default.
- Ciclos for, while, do-while.

- println(...), print(...), readln().

10.3 Lenguaje .pig (Pig Latin)

Lenguaje principal que integra los otros dos. Aquí está el main.

Ejemplo:

```
pig
import Persona.z
import Funciones.y
```

VARIABLES>

```
esto edad : numerus 25;
esto nombre : textum "Juan";
```

MAIOR>

```
>> "Hola, ";
>> nombre;
>> "Tu edad es: ";
>> edad;
```

```
esto p : novus Persona("Ana", 30);
>> p.getNombre();
```

FINIS;

Características:

- Imports de .y y .z.
 - Sección de variables globales (VARIABLES>).
 - Sección principal (MAIOR>).
 - Cierre con FINIS;.
 - Instanciación de objetos con novus.
 - Llamadas a funciones de .y y métodos de .z.
 - Condicionales si / aliter / aliter.
 - Ciclos dum, facere, per.
 - perge (continue) e interrompe (break).
 - >> para imprimir y << para leer.
-

11. Ejemplos completos

11.1 Ejemplo mínimo (.pig sin imports)

Archivo Hola.pig:

```
pig
MAIOR>
>> "Hola, mundo!";
FINIS;
```

Pasos:

1. Guarda el archivo.
2. Pulsa F6 (Compilar).

3. Elige dónde guardar Hola.c.
4. Abre una terminal, ve a esa carpeta y ejecuta:
5. `bash`
6. `./Hola`
7. Salida esperada:
8. `text`
9. Hola, mundo!

11.2 Ejemplo con función de .y

Archivo Funciones.y:

`y`

`%funciones`

definir sumar(entero a, entero b) -> entero:

 retornar a + b

Archivo Programa.pig:

`pig`

`import Funciones.y`

MAIOR>

esto resultado : numerus sumar(10, 20);

>> "Resultado: ";

>> resultado;

FINIS;

Salida esperada:

`text`

Resultado: 30

11.3 Ejemplo con clase de .z

Archivo Persona.z:

`java`

`public class Persona {`

 String nombre;

 int edad;

 public Persona(String nombreParametro, int edadParametro) {

 nombre = nombreParametro;

 edad = edadParametro;

 }

 public String getNombre() {

 return nombre;

 }

`}`

Archivo Programa.pig:

`pig`

`import Persona.z`

```
MAIOR>
esto p : novus Persona("Ana", 25);
>> "Nombre: ";
>> p.getNombre();
FINIS;
Salida esperada:
text
Nombre: Ana
```

12. Atajos de teclado

Atajo	Acción
Ctrl + N	Nuevo archivo
Ctrl + O	Abrir archivo
Ctrl + Shift + O	Abrir carpeta
Ctrl + S	Guardar
Ctrl + Shift + S	Guardar como
F5	Analizar
F6	Compilar
Ctrl + Q	Salir
Ctrl + Z	Deshacer
Ctrl + Y	Rehacer
Ctrl + X	Cortar
Ctrl + C	Copiar
Ctrl + V	Pegar

13. Preguntas frecuentes

13.1 ¿Por qué no puedo compilar un archivo .y o .z?

Porque solo .pig tiene main y puede generar un ejecutable. Los otros dos son lenguajes de apoyo que se importan desde .pig.

13.2 ¿Por qué aparece "Variable no declarada"?

Porque usaste una variable que no está declarada antes de su uso. En los 3 lenguajes, las variables deben declararse antes de usarse.

13.3 ¿Por qué aparece "Tipo incompatible"?

Porque intentas asignar un valor de un tipo a una variable de otro tipo incompatible.

Ejemplo:

pig

esto x : numerus "hola"; //  numero recibe texto

13.4 ¿Por qué el programa compila pero al ejecutarlo no pasa nada?

Puede ser porque:

- El .pig no imprime nada.
- El .pig termina inmediatamente.
- El programa se queda esperando un << en la terminal.

13.5 ¿Por qué no se genera el ejecutable?

Porque GCC no está instalado o no está en el PATH. Verifica con gcc --version en una terminal.

13.6 ¿Por qué el editor no resalta mi código?

Porque el archivo no tiene una extensión reconocida. Debe ser .y, .z o .pig.

13.7 ¿Dónde se guarda el código C?

Donde tú elijas en el diálogo al compilar. Por defecto se sugiere la carpeta del .pig con el nombre del archivo cambiado a .c.

13.8 ¿Dónde se guarda el ejecutable?

Al lado del .c, con el mismo nombre sin extensión. Ejemplo: si el .c es /home/user/Programa.c, el ejecutable es /home/user/Programa.

13.9 ¿Puedo ejecutar el programa desde la aplicación?

No. Debes abrir una terminal y ejecutar el binario manualmente. Esto es intencional para permitir la entrada de datos con <<.

13.10 ¿Cómo detengo un programa que se queda colgado?

Cierra la terminal con Ctrl + C (Linux/macOS) o Ctrl + C / Ctrl + Break (Windows).

14. Cierre

Este manual cubre el uso básico de la aplicación. Para detalles sobre el diseño interno, la gramática de los lenguajes y las tecnologías usadas, consulta el Manual Técnico.

Buen código, cadete.